Looks pretty straightforward, right? The answer should be *8* in a sane world? Well on most systems the result of that calculation is actually something like `float(7.999999999999999)`. Which is definitely very close to *8*, so we could give it a pass? How about this:

```
var_dump(floor((0.1 + 0.7) * 10));
```

```
Output:
float(7)
```

This error arises because certain rational decimal numbers (that have a finite number of decimal digits, unlike, say, *pi*) cannot be accurately represented with floats, because the binary equivalent is irrational. *0.1* and *0.7* are two of them. The binary representation of these numbers is only approximate. As more operations are performed with floats, the possibility grows of accumulated precision errors. These often become most apparent during comparison:

```
$bankBalance = 50.00;
$bankBalance -= 49.95;
$bankBalance -= 0.05;

if ($bankBalance < 0.0) {
    echo "Overdraft fees!" . PHP_EOL;
}

var_dump($bankBalance);
```

```
Output:
Overdraft fees!
float(-2.8449465006019636E-15)
```

In the imaginary scenario above, we have some code that uses floats to maintain a bank balance and a test to see if the balance drops below zero. Despite having enough funds to cover the two debits, we still fall into the overdraft fees trap. And this is why financial apps tend to work internally with integer values of the smallest currency units.

We can summarize the problems that IEEE floats have as follows:

- Significant digits are limited; therefore, all irrational numbers (like *1/3* or *pi*) are rounded to an approximation